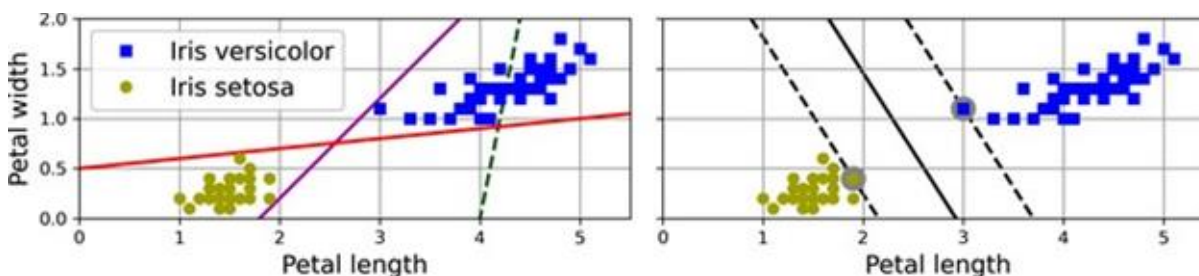


CHƯƠNG 5. MÁY HỌC VEC-TƠ HỖ TRỢ

Một máy học véc-tơ hỗ trợ (SVM) là một mô hình học máy mạnh mẽ và linh hoạt, có khả năng thực hiện phân loại tuyến tính hoặc phi tuyến, hồi quy, và thậm chí cả phát hiện điểm mới. SVM đặc biệt hiệu quả với các tập dữ liệu phi tuyến có kích thước nhỏ đến trung bình (tức là hàng trăm đến hàng nghìn trường hợp), đặc biệt cho các tác vụ phân loại. Tuy nhiên, chúng không mở rộng tốt với các tập dữ liệu rất lớn, như bạn sẽ thấy. Chương này sẽ giải thích các khái niệm cốt lõi của SVM, cách sử dụng chúng và cách chúng hoạt động. Hãy cùng tìm hiểu ngay!

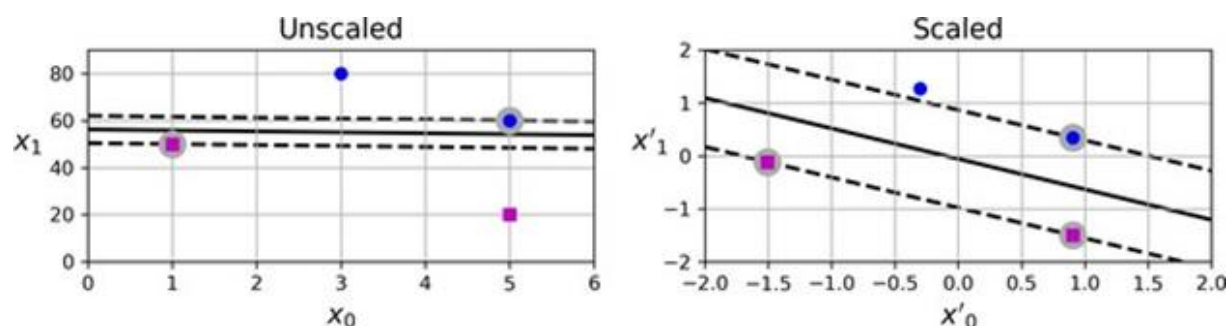
5.1. Phân loại SVM tuyến tính

Ý tưởng cơ bản đằng sau SVM được giải thích tốt nhất bằng một số hình ảnh. Hình 5-1 cho thấy một phần của tập dữ liệu iris đã được giới thiệu ở cuối Chương 4. Hai lớp có thể được phân tách rõ ràng một cách dễ dàng bằng một đường thẳng (chúng có thể phân tách tuyến tính). Biểu đồ bên trái cho thấy các đường biên quyết định của ba bộ phân loại tuyến tính có thể có. Mô hình có đường biên quyết định được biểu thị bằng đường nét đứt tẻ đến mức nó thậm chí không phân tách các lớp một cách đúng đắn. Hai mô hình còn lại hoạt động hoàn hảo trên tập huấn luyện này, nhưng các đường biên quyết định của chúng quá gần với các trường hợp khiến các mô hình này có thể không hoạt động tốt trên các trường hợp mới. Ngược lại, đường nét liền trong biểu đồ bên phải đại diện cho đường biên quyết định của một bộ phân loại SVM; đường này không chỉ phân tách hai lớp mà còn giữ khoảng cách xa nhất có thể so với các trường hợp huấn luyện gần nhất. Bạn có thể coi một bộ phân loại SVM như việc khớp một “con đường” rộng nhất có thể (được biểu thị bằng các đường nét đứt song song) giữa các lớp. Đây được gọi là phân loại biên lớn (large margin classification).



Hình 5-1. Phân loại biên lớn

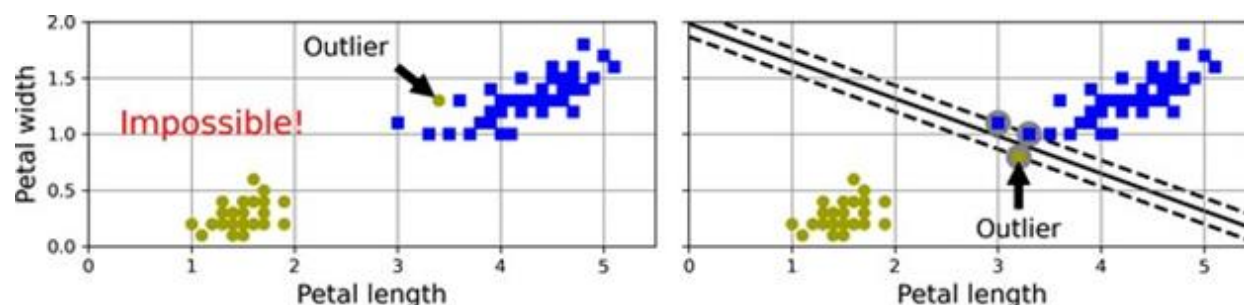
Lưu ý rằng việc thêm nhiều trường hợp huấn luyện “ngoài lề” sẽ không ảnh hưởng đến đường biên quyết định: nó hoàn toàn được xác định (hoặc “hỗ trợ”) bởi các trường hợp nằm trên rìa của “con đường”. Các trường hợp này được gọi là các véc-tơ hỗ trợ (chúng được khoanh tròn trong Hình 5-1).



Hình 5-2. Độ nhạy với thang đo đặc trưng

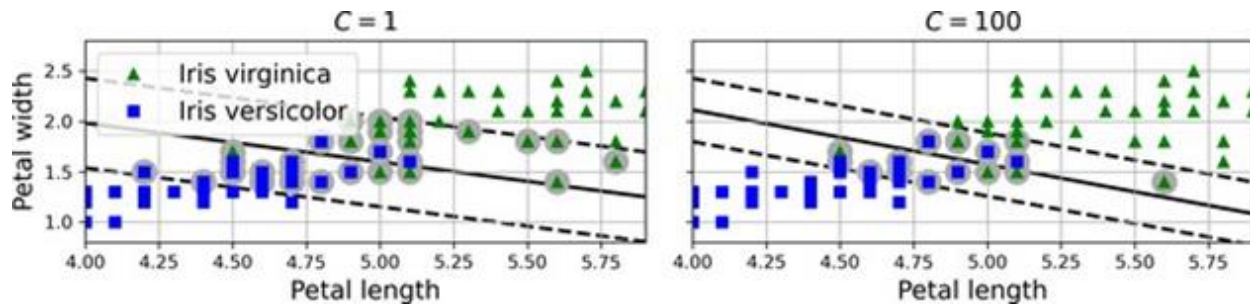
5.1.1 Phân loại biên mềm

Nếu chúng ta áp đặt nghiêm ngặt rằng tất cả các trường hợp phải nằm ngoài “con đường” và ở đúng phía, đây được gọi là phân loại biên cứng (hard margin classification). Có hai vấn đề chính với phân loại biên cứng. Thứ nhất, nó chỉ hoạt động nếu dữ liệu có thể phân tách tuyến tính. Thứ hai, nó nhạy cảm với các ngoại lệ. Hình 5-3 cho thấy tập dữ liệu iris chỉ với một ngoại lệ bổ sung: ở bên trái, không thể tìm thấy biên cứng; ở bên phải, đường biên quyết định rất khác so với đường chúng ta thấy trong Hình 5-1 mà không có ngoại lệ, và mô hình có thể sẽ không tổng quát hóa tốt.



Hình 5-3. Độ nhạy của biên cứng với các ngoại lệ

Để tránh những vấn đề này, chúng ta cần sử dụng một mô hình linh hoạt hơn. Mục tiêu là tìm sự cân bằng tốt giữa việc giữ “con đường” càng lớn càng tốt và hạn chế các vi phạm biên (tức là các trường hợp nằm ở giữa “con đường” hoặc thậm chí ở sai phía). Đây được gọi là phân loại biên mềm (soft margin classification). Khi tạo một mô hình SVM bằng Scikit-Learn, bạn có thể chỉ định một số siêu tham số, bao gồm siêu tham số chính quy hóa C . Nếu bạn đặt nó thành giá trị thấp, bạn sẽ nhận được mô hình ở bên trái Hình 5-4. Với giá trị cao, bạn sẽ nhận được mô hình ở bên phải. Như bạn có thể thấy, giảm C làm cho “con đường” lớn hơn, nhưng nó cũng dẫn đến nhiều vi phạm biên hơn. Nói cách khác, giảm C dẫn đến nhiều trường hợp hỗ trợ “con đường” hơn, do đó ít rủi ro về việc quá khớp (overfitting). Nhưng nếu bạn giảm nó quá nhiều, mô hình sẽ bị dưới khớp (underfitting), như trường hợp này: mô hình với $C=100$ dường như sẽ tổng quát hóa tốt hơn mô hình với $C=1$.



Hình 5-4. Biên lớn (trái) so với ít vi phạm biên hơn (phải)

Đoạn mã Scikit-Learn sau đây tải tập dữ liệu iris và huấn luyện một bộ phân loại SVM tuyến tính để phát hiện hoa Iris virginica. Pipeline đầu tiên chuẩn hóa các đặc trưng, sau đó sử dụng LinearSVC với $C=1$:

```
from sklearn.datasets import load_iris
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

# Tải dữ liệu Iris
iris = load_iris(as_frame=True)

# Chọn đặc trưng và nhãn
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = (iris.target == 2) # Nhãn cho Iris virginica

# Tạo pipeline với StandardScaler và LinearSVC
svm_clf = make_pipeline(
    StandardScaler(),
    LinearSVC(C=1, random_state=42)
)

# Huấn luyện mô hình
svm_clf.fit(X, y)
```

Mô hình thu được được biểu thị ở bên trái trong Hình 5-4. Sau đó, như thường lệ, bạn có thể sử dụng mô hình để đưa ra dự đoán:

```
X_new = [[5.5, 1.7], [5.0, 1.5]] svm_clf.predict(X_new) array([ True, False])
```

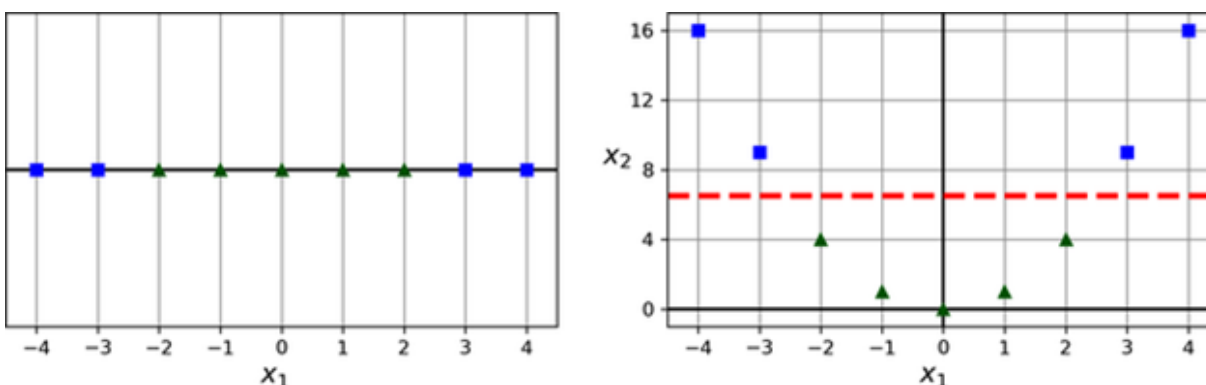
Cây đầu tiên được phân loại là Iris virginica, trong khi cây thứ hai thì không. Hãy xem xét các điểm mà SVM đã sử dụng để đưa ra các dự đoán này. Chúng đo khoảng cách có dấu giữa mỗi trường hợp và đường biên quyết định:

```
svm_clf.decision_function(X_new) array([ 0.66163411, -0.22036063])
```

Không giống như LogisticRegression, LinearSVC không có phương thức `predict_proba()` để ước tính xác suất lớp. Điều đó nói lên rằng, nếu bạn sử dụng lớp SVC (sẽ được thảo luận sau) thay vì LinearSVC, và nếu bạn đặt siêu tham số `probability` của nó thành `True`, thì mô hình sẽ khớp một mô hình bổ sung ở cuối quá trình huấn luyện để ánh xạ các điểm chức năng quyết định của SVM sang xác suất ước tính. Về cơ bản, điều này yêu cầu sử dụng xác thực chéo 5 lần để tạo ra các dự đoán ngoài mẫu cho mọi trường hợp trong tập huấn luyện, sau đó huấn luyện một mô hình LogisticRegression, do đó sẽ làm chậm quá trình huấn luyện đáng kể. Sau đó, các phương thức `predict_proba()` và `predict_log_proba()` sẽ có sẵn.

Phân loại SVM phi tuyến

Mặc dù các bộ phân loại SVM tuyến tính hiệu quả và thường hoạt động đáng ngạc nhiên, nhiều tập dữ liệu thậm chí còn không gần với việc có thể phân tách tuyến tính. Một cách tiếp cận để xử lý các tập dữ liệu phi tuyến là thêm nhiều đặc trưng hơn, chẳng hạn như các đặc trưng đa thức (như chúng ta đã làm trong Chương 4); trong một số trường hợp, điều này có thể dẫn đến một tập dữ liệu có thể phân tách tuyến tính. Hãy xem xét biểu đồ bên trái trong Hình 5-5: nó biểu thị một tập dữ liệu đơn giản chỉ với một đặc trưng, x_1 . Tập dữ liệu này không thể phân tách tuyến tính, như bạn có thể thấy. Nhưng nếu bạn thêm một đặc trưng thứ hai $x_2 = (x_1)^2$, tập dữ liệu 2D thu được hoàn toàn có thể phân tách tuyến tính.



Hình 5-5. Thêm đặc trưng để làm cho một tập dữ liệu có thể phân tách tuyến tính

Để triển khai ý tưởng này bằng Scikit-Learn, bạn có thể tạo một pipeline chứa bộ biến đổi PolynomialFeatures (được thảo luận trong “Hồi quy đa thức”), theo sau là StandardScaler và một bộ phân loại LinearSVC. Hãy thử nghiệm điều này trên tập dữ liệu moons, một tập dữ liệu đồ chơi để phân loại nhị phân trong đó các điểm dữ liệu có hình dạng hai lưỡi liềm xen kẽ (xem Hình 5-6). Bạn có thể tạo tập dữ liệu này bằng cách sử dụng hàm `make_moons()`:

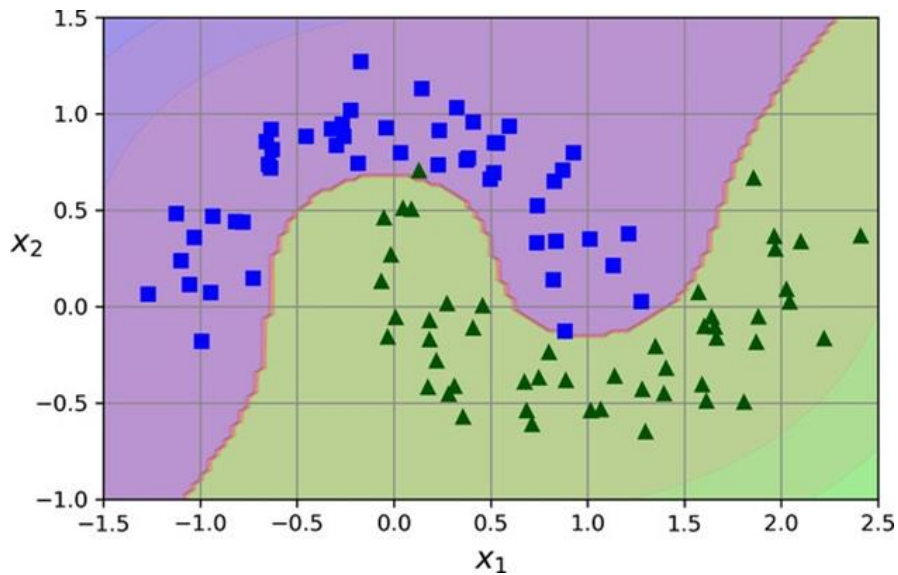
```
from sklearn.datasets import make_moons
from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
from sklearn.pipeline import make_pipeline
```

```
# Tạo dữ liệu dạng hình mặt trăng
```

```
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
```

```
# Tạo pipeline: thêm đặc trưng đa thức bậc 3, chuẩn hóa, sau đó huấn luyện SVM tuyến tính
polynomial_svm_clf = make_pipeline(
    PolynomialFeatures(degree=3),
    StandardScaler(),
    LinearSVC(C=10, max_iter=10_000, random_state=42)
)
```

```
# Huấn luyện mô hình
polynomial_svm_clf.fit(X, y)
```



Hình 5-6. Bộ phân loại SVM tuyến tính sử dụng các đặc trưng đa thức

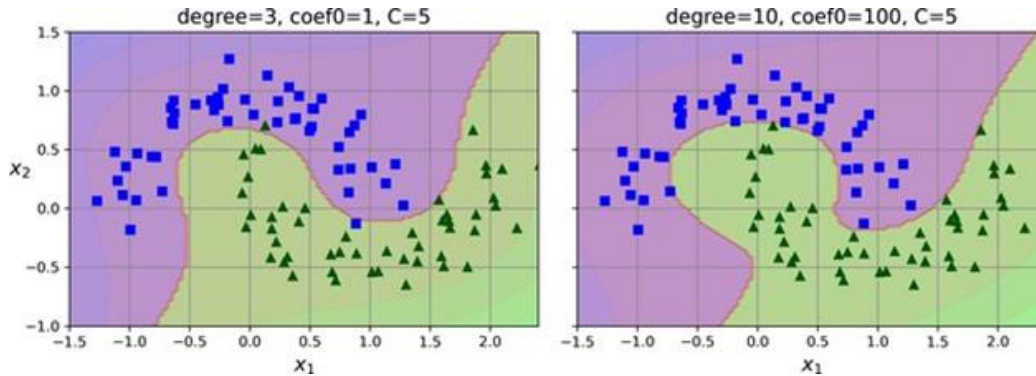
Kernel đa thức

Việc thêm các đặc trưng đa thức rất đơn giản để triển khai và có thể hoạt động tuyệt vời với tất cả các loại thuật toán học máy (không chỉ SVM). Điều đó nói lên rằng, ở bậc đa thức thấp, phương pháp này không thể xử lý các tập dữ liệu rất phức tạp, và ở bậc đa thức cao, nó tạo ra một số lượng lớn các đặc trưng, làm cho mô hình quá chậm. May mắn thay, khi sử dụng SVM, bạn có thể áp dụng một kỹ thuật toán học gần như kỳ diệu được gọi là kernel trick (sẽ được giải thích sau trong chương này). Kernel trick giúp có thể đạt được kết quả tương tự như khi bạn đã thêm nhiều đặc trưng đa thức, ngay cả với bậc rất cao, mà không thực sự phải thêm chúng. Điều này có nghĩa là không có sự bùng nổ tổ hợp về số lượng đặc trưng. Thủ thuật này được triển khai bởi lớp SVC. Hãy thử nghiệm nó trên tập dữ liệu moons:

SVC

```
poly_kernel_svm_clf = make_pipeline(StandardScaler(), SVC(kernel="poly", degree=3,
coef0=1, C=5)) poly_kernel_svm_clf.fit(X, y)
```

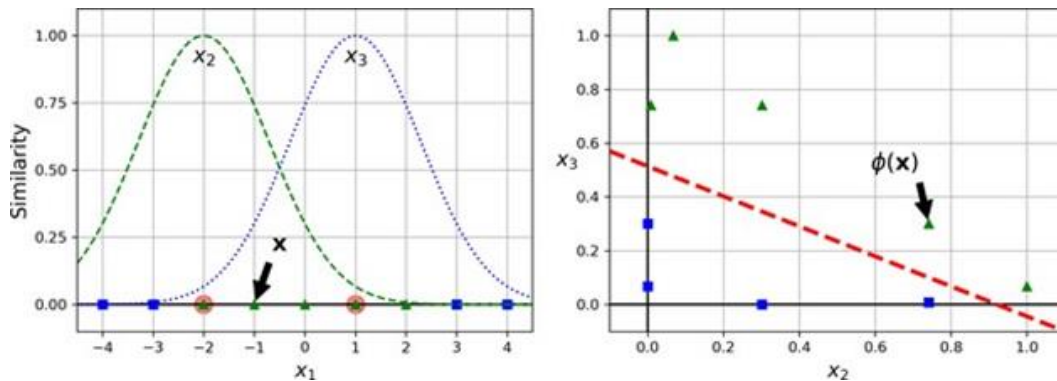

Đoạn mã này huấn luyện một bộ phân loại SVM sử dụng kernel đa thức bậc ba, được biểu thị ở bên trái trong Hình 5-7. Ở bên phải là một bộ phân loại SVM khác sử dụng kernel đa thức bậc 10. Rõ ràng, nếu mô hình của bạn đang bị quá khớp, bạn có thể muốn giảm bậc đa thức. Ngược lại, nếu nó bị dưới khớp, bạn có thể thử tăng nó. Siêu tham số coef0 kiểm soát mức độ ảnh hưởng của các số hạng bậc cao so với các số hạng bậc thấp đến mô hình.



Hình 5-7. Các bộ phân loại SVM với kernel đa thức

Các đặc trưng tương tự

Một kỹ thuật khác để giải quyết các vấn đề phi tuyến là thêm các đặc trưng được tính toán bằng hàm tương tự (similarity function), đo lường mức độ mỗi trường hợp giống một điểm mốc cụ thể, như chúng ta đã làm trong Chương 2 khi thêm các đặc trưng tương tự địa lý. Ví dụ, hãy lấy tập dữ liệu 1D từ trước và thêm hai điểm mốc vào đó tại $x_1 = -2$ và $x_1 = 1$ (xem biểu đồ bên trái trong Hình 5-8). Tiếp theo, chúng ta sẽ định nghĩa hàm tương tự là Gaussian RBF với $\gamma = 0.3$. Đây là một hàm hình chuông thay đổi từ 0 (rất xa điểm mốc) đến 1 (tại điểm mốc). Bây giờ chúng ta đã sẵn sàng để tính toán các đặc trưng mới. Ví dụ, hãy xem xét trường hợp $x_1 = -1$: nó nằm ở khoảng cách 1 từ điểm mốc đầu tiên và 2 từ điểm mốc thứ hai. Do đó, các đặc trưng mới của nó là $x_2 = \exp(-0.3 \times 1^2) \approx 0.74$ và $x_3 = \exp(-0.3 \times 2^2) \approx 0.30$. Biểu đồ bên phải trong Hình 5-8 cho thấy tập dữ liệu đã được biến đổi (bỏ qua các đặc trưng gốc). Như bạn có thể thấy, nó hiện có thể phân tách tuyến tính.



Hình 5-8. Các đặc trưng tương tự sử dụng Gaussian RBF

Bạn có thể tự hỏi làm thế nào để chọn các điểm mốc. Cách tiếp cận đơn giản nhất là tạo một điểm mốc tại vị trí của mỗi và mọi trường hợp trong tập dữ liệu. Làm như vậy tạo ra nhiều

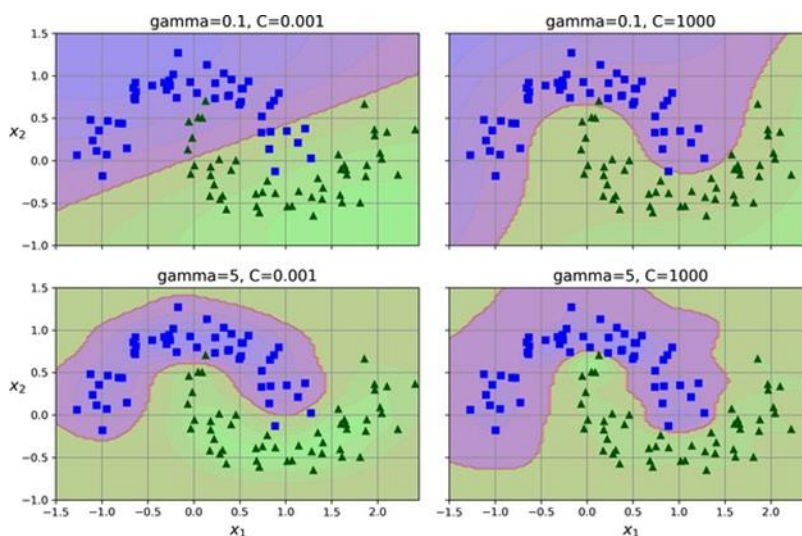
chiều và do đó tăng khả năng tập huấn luyện đã được biến đổi sẽ có thể phân tách tuyến tính. Nhược điểm là một tập huấn luyện với m trường hợp và n đặc trưng sẽ được biến đổi thành một tập huấn luyện với m trường hợp và m đặc trưng (giả sử bạn bỏ qua các đặc trưng gốc). Nếu tập huấn luyện của bạn rất lớn, bạn sẽ có một số lượng đặc trưng tương đương.

Kernel Gaussian RBF

Cũng giống như phương pháp đặc trưng đa thức, phương pháp đặc trưng tương tự có thể hữu ích với bất kỳ thuật toán học máy nào, nhưng việc tính toán tất cả các đặc trưng bổ sung có thể tốn kém về mặt tính toán (đặc biệt trên các tập huấn luyện lớn). Một lần nữa, kernel trick lại thực hiện phép thuật SVM của nó, giúp có thể đạt được kết quả tương tự như khi bạn đã thêm nhiều đặc trưng tương tự, nhưng không thực sự làm như vậy. Hãy thử lớp SVC với kernel Gaussian RBF:

```
rbf_kernel_svm_clf = make_pipeline(StandardScaler(), SVC(kernel="rbf", gamma=5, C=0.001))
rbf_kernel_svm_clf.fit(X, y)
```

Mô hình này được biểu thị ở phía dưới bên trái trong Hình 5-9. Các biểu đồ khác cho thấy các mô hình được huấn luyện với các giá trị khác nhau của các siêu tham số γ và C . Tăng γ làm cho đường cong hình chuông hẹp hơn (xem các biểu đồ bên trái trong Hình 5-8). Kết quả là, phạm vi ảnh hưởng của mỗi trường hợp nhỏ hơn: đường biên quyết định trở nên bất thường hơn, uốn lượn quanh từng trường hợp riêng lẻ. Ngược lại, giá trị γ nhỏ làm cho đường cong hình chuông rộng hơn: các trường hợp có phạm vi ảnh hưởng lớn hơn, và đường biên quyết định trở nên mượt mà hơn. Vì vậy, γ hoạt động giống như một siêu tham số chính quy hóa: nếu mô hình của bạn bị quá khớp, bạn nên giảm γ ; nếu nó bị dưới khớp, bạn nên tăng γ (tương tự như siêu tham số C).



Hình 5-9. Các bộ phân loại SVM sử dụng kernel RBF

Các kernel khác tồn tại nhưng được sử dụng ít thường xuyên hơn nhiều. Một số kernel được chuyên biệt cho các cấu trúc dữ liệu cụ thể. Các kernel chuỗi đôi khi được sử dụng khi phân

loại tài liệu văn bản hoặc chuỗi DNA (ví dụ: sử dụng kernel chuỗi con hoặc các kernel dựa trên khoảng cách Levenshtein).

Các lớp SVM và độ phức tạp tính toán

Lớp LinearSVC dựa trên thư viện liblinear, triển khai một thuật toán tối ưu hóa cho SVM tuyến tính. Nó không hỗ trợ kernel trick, nhưng nó mở rộng gần như tuyến tính với số lượng trường hợp huấn luyện và số lượng đặc trưng. Độ phức tạp thời gian huấn luyện của nó xấp xỉ $O(m \times n)$. Thuật toán mất nhiều thời gian hơn nếu bạn yêu cầu độ chính xác rất cao. Điều này được kiểm soát bởi siêu tham số dung sai ϵ (được gọi là tol trong Scikit-Learn). Trong hầu hết các tác vụ phân loại, dung sai mặc định là tốt. Lớp SVC dựa trên thư viện libsvm, triển khai một thuật toán hỗ trợ kernel trick. Độ phức tạp thời gian huấn luyện thường nằm trong khoảng từ $O(m^2 \times n)$ đến $O(m^3 \times n)$. Thật không may, điều này có nghĩa là nó trở nên cực kỳ chậm khi số lượng trường hợp huấn luyện trở nên lớn (ví dụ: hàng trăm nghìn trường hợp), vì vậy thuật toán này tốt nhất cho các tập huấn luyện phi tuyến có kích thước nhỏ hoặc trung bình. Nó mở rộng tốt với số lượng đặc trưng, đặc biệt với các đặc trưng thưa thớt (tức là khi mỗi trường hợp có ít đặc trưng khác không). Trong trường hợp này, thuật toán mở rộng xấp xỉ với số lượng trung bình các đặc trưng khác không trên mỗi trường hợp. Lớp SGDClassifier cũng thực hiện phân loại biên lớn theo mặc định, và các siêu tham số của nó – đặc biệt là các siêu tham số chính quy hóa (alpha và penalty) và learning_rate – có thể được điều chỉnh để tạo ra kết quả tương tự như SVM tuyến tính. Để huấn luyện, nó sử dụng giảm độ dốc ngẫu nhiên (xem Chương 4), cho phép học tăng dần và sử dụng ít bộ nhớ, vì vậy bạn có thể sử dụng nó để huấn luyện một mô hình trên một tập dữ liệu lớn không phù hợp với RAM (tức là cho học ngoài lõi). Hơn nữa, nó mở rộng rất tốt, vì độ phức tạp tính toán của nó là $O(m \times n)$. Bảng 5-1 so sánh các lớp phân loại SVM của Scikit-Learn.

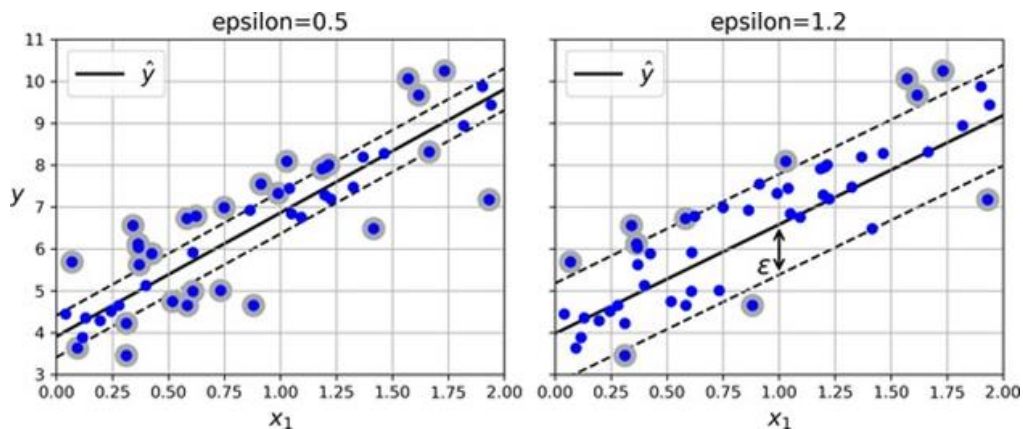
Bảng 5-1. So sánh các lớp Scikit-Learn cho phân loại SVM

Class	Time complexity	Out-of-core support	Scaling required	Kernel trick
LinearSVC	$O(m \times n)$	No	Yes	No
SVC	$O(m^2 \times n)$ to $O(m^3 \times n)$	No	Yes	Yes
SGDClassifier	$O(m \times n)$	Yes	Yes	No

Bây giờ chúng ta hãy xem các thuật toán SVM cũng có thể được sử dụng cho hồi quy tuyến tính và phi tuyến như thế nào.

Hồi quy SVM

Để sử dụng SVM cho hồi quy thay vì phân loại, mẹo là điều chỉnh mục tiêu: thay vì cố gắng khớp “con đường” lớn nhất có thể giữa hai lớp trong khi hạn chế vi phạm biên, hồi quy SVM cố gắng khớp càng nhiều trường hợp càng tốt trên “con đường” trong khi hạn chế vi phạm biên (tức là các trường hợp nằm ngoài “con đường”). Chiều rộng của “con đường” được kiểm soát bởi một siêu tham số, ϵ . Hình 5-10 cho thấy hai mô hình hồi quy SVM tuyến tính được huấn luyện trên một số dữ liệu tuyến tính, một với biên nhỏ ($\epsilon = 0.5$) và một với biên lớn hơn ($\epsilon = 1.2$).



Hình 5-10. Hồi quy SVM

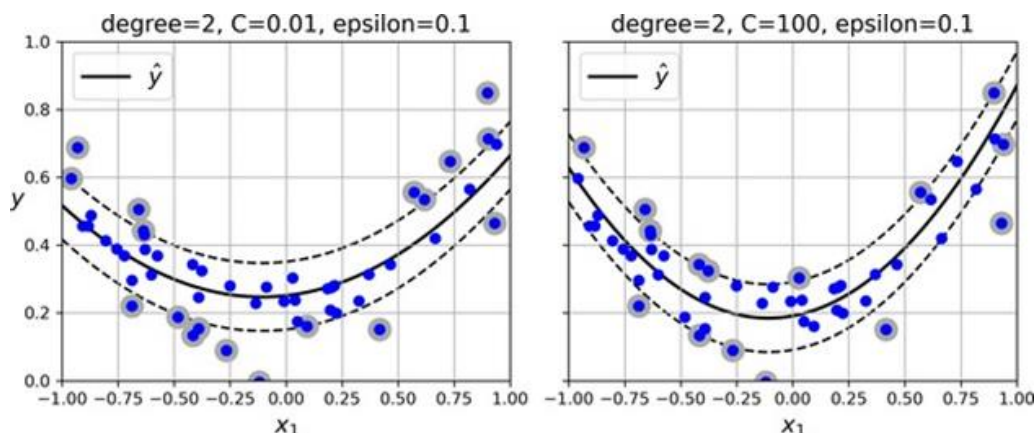
Giảm ϵ làm tăng số lượng véc-tơ hỗ trợ, điều này chuẩn hóa mô hình. Hơn nữa, nếu bạn thêm nhiều trường hợp huấn luyện hơn trong biên, nó sẽ không ảnh hưởng đến dự đoán của mô hình; do đó, mô hình được gọi là không nhạy cảm với ϵ .

Bạn có thể sử dụng lớp LinearSVR của Scikit-Learn để thực hiện hồi quy SVM tuyến tính. Đoạn mã sau đây tạo ra mô hình được biểu thị ở bên trái trong Hình 5-10:

LinearSVR

```
X, y = [...] # một tập dữ liệu tuyến tính
svm_reg = make_pipeline(StandardScaler(),
LinearSVR(epsilon=0.5, random_state=42))
svm_reg.fit(X, y)
```

Để giải quyết các tác vụ hồi quy phi tuyến, bạn có thể sử dụng mô hình SVM với kernel. Hình 5-11 cho thấy hồi quy SVM trên một tập huấn luyện bậc hai ngẫu nhiên, sử dụng kernel đa thức bậc hai. Có một số chính quy hóa trong biểu đồ bên trái (tức là giá trị C nhỏ), và ít hơn nhiều ở biểu đồ bên phải (tức là giá trị C lớn).



Hình 5-11. Hồi quy SVM sử dụng kernel đa thức bậc hai

Đoạn mã sau đây sử dụng lớp SVR của Scikit-Learn (hỗ trợ kernel trick) để tạo ra mô hình được biểu thị ở bên trái trong Hình 5-11:

SVR

$X, y = [...]$ # một tập dữ liệu bậc hai `svm_poly_reg = make_pipeline(StandardScaler(), SVR(kernel="poly", degree=2, C=0.01, epsilon=0.1)) svm_poly_reg.fit(X, y)`

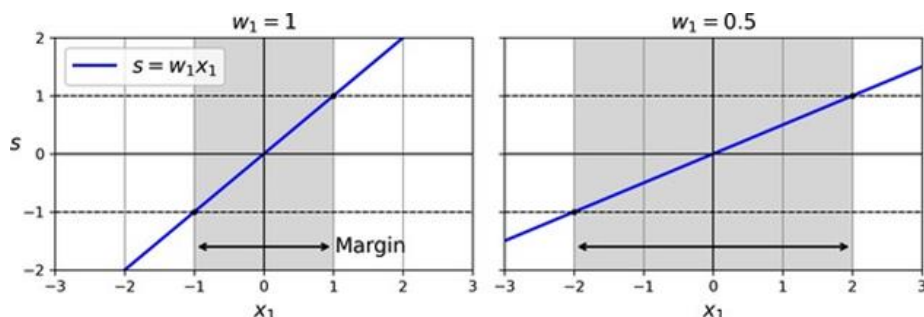
Lớp SVR là tương đương hồi quy của lớp SVC, và lớp LinearSVR là tương đương hồi quy của lớp LinearSVC. Lớp LinearSVR mở rộng tuyến tính với kích thước của tập huấn luyện (giống như lớp LinearSVC), trong khi lớp SVR trở nên quá chậm khi tập huấn luyện trở nên rất lớn (giống như lớp SVC).

Phần còn lại của chương này giải thích cách SVM đưa ra dự đoán và cách các thuật toán huấn luyện của chúng hoạt động, bắt đầu với các bộ phân loại SVM tuyến tính. Nếu bạn mới bắt đầu với học máy, bạn có thể bỏ qua phần này và đi thẳng đến các bài tập ở cuối chương này, và quay lại sau khi bạn muốn hiểu sâu hơn về SVM.

Bên trong các bộ phân loại SVM tuyến tính

Bộ phân loại SVM tuyến tính dự đoán lớp của một trường hợp mới x bằng cách đầu tiên tính toán hàm quyết định $\theta^T x = \theta_0 x_0 + \dots + \theta_n x_n$, trong đó x_0 là đặc trưng bias (luôn bằng 1). Nếu kết quả là dương, thì lớp dự đoán y là lớp dương (1); nếu không thì là lớp âm (0). Điều này giống hệt như Hồi quy Logistic (được thảo luận trong Chương 4).

Dự đoán bằng bộ phân loại SVM tuyến tính khá đơn giản. Vậy còn việc huấn luyện thì sao? Điều này đòi hỏi phải tìm vector trọng số w và số hạng bias b sao cho “con đường”, hay biên, rộng nhất có thể trong khi hạn chế số lượng vi phạm biên. Hãy bắt đầu với chiều rộng của “con đường”: để làm cho nó lớn hơn, chúng ta cần làm cho w nhỏ hơn. Điều này có thể dễ hình dung hơn trong không gian 2D, như trong Hình 5-12. Hãy định nghĩa các đường biên của “con đường” là các điểm mà hàm quyết định bằng -1 hoặc $+1$. Trong biểu đồ bên trái, trọng số w_1 là 1, vì vậy các điểm mà $w_1 x_1 = -1$ hoặc $+1$ là $x_1 = -1$ và $+1$: do đó kích thước của biên là 2. Trong biểu đồ bên phải, trọng số là 0.5, vì vậy các điểm mà $w_1 x_1 = -1$ hoặc $+1$ là $x_1 = -2$ và $+2$: kích thước của biên là 4. Vì vậy, chúng ta cần giữ w càng nhỏ càng tốt. Lưu ý rằng số hạng bias b không ảnh hưởng đến kích thước của biên: việc điều chỉnh nó chỉ làm dịch chuyển biên mà không ảnh hưởng đến kích thước của nó.



Hình 5-12. Một vector trọng số nhỏ hơn dẫn đến một biên lớn hơn

Chúng ta cũng muốn tránh các vi phạm biên, vì vậy chúng ta cần hàm quyết định lớn hơn 1 cho tất cả các trường hợp huấn luyện dương và nhỏ hơn -1 cho các trường hợp huấn luyện

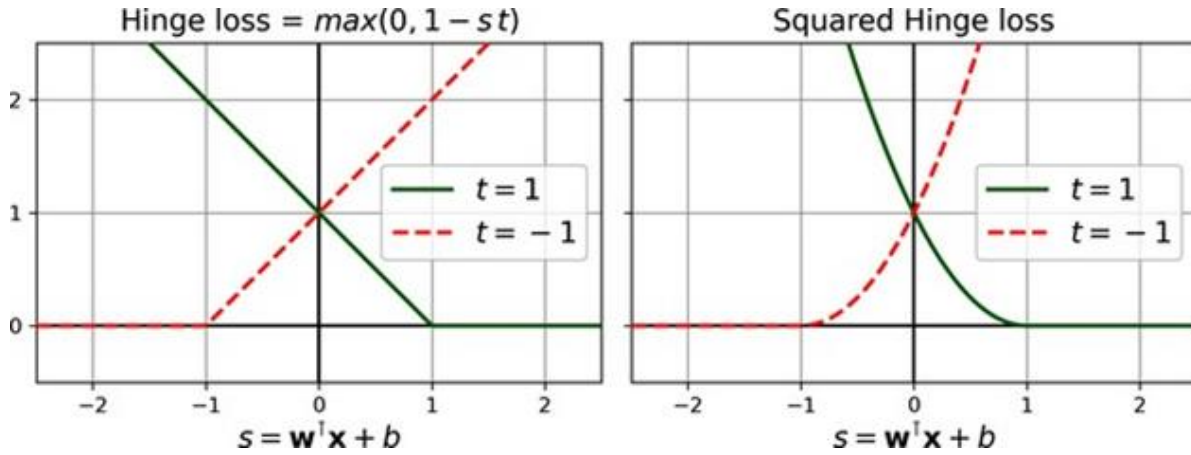
âm. Nếu chúng ta định nghĩa $t^{(i)} = -1$ cho các trường hợp âm (khi $y^{(i)} = 0$) và $t^{(i)} = 1$ cho các trường hợp dương (khi $y^{(i)} = 1$), thì chúng ta có thể viết ràng buộc này là $t^{(i)}(w^T x^{(i)} + b) \geq 1$ cho tất cả các trường hợp. Do đó, chúng ta có thể biểu diễn mục tiêu của bộ phân loại SVM tuyến tính biên cứng dưới dạng bài toán tối ưu hóa có ràng buộc trong Phương trình 5-1. Phương trình 5-1. Mục tiêu của bộ phân loại SVM tuyến tính biên cứng tối thiểu hóa w, b

$$\begin{aligned} & \underset{w, b}{\text{minimize}} \quad \frac{1}{2} w^T w \\ & \text{subject to} \quad t^{(i)}(w^T x^{(i)} + b) \geq 1 \text{ với } i = 1, 2, \dots, m \end{aligned}$$

Để có được mục tiêu biên mềm, chúng ta cần đưa vào một biến trượt $\zeta^{(i)} \geq 0$ cho mỗi trường hợp: $\zeta^{(i)}$ đo lường mức độ trường hợp thứ i được phép vi phạm biên. Bây giờ chúng ta có hai mục tiêu xung đột: làm cho các biến trượt càng nhỏ càng tốt để giảm vi phạm biên, và làm cho $\frac{1}{2} w^T w$ càng nhỏ càng tốt để tăng biên. Đây là lúc siêu tham số C xuất hiện: nó cho phép chúng ta định nghĩa sự cân bằng giữa hai mục tiêu này. Điều này cho chúng ta bài toán tối ưu hóa có ràng buộc trong Phương trình 5-2. Phương trình 5-2. Mục tiêu của bộ phân loại SVM tuyến tính biên mềm tối thiểu hóa w, b, ζ

$$\begin{aligned} & \frac{1}{2} w^T w + C \sum_{i=1}^m \zeta^{(i)} \\ & \text{thỏa mãn } t^{(i)}(w^T x^{(i)} + b) \geq 1 - \zeta^{(i)} \text{ và } \zeta^{(i)} \geq 0 \text{ cho } i = 1, 2, \dots, m \end{aligned}$$

Cả bài toán biên cứng và biên mềm đều là các bài toán tối ưu hóa bậc hai lồi với các ràng buộc tuyến tính. Các bài toán như vậy được gọi là bài toán quy hoạch bậc hai (QP). Nhiều bộ giải có sẵn để giải các bài toán QP bằng cách sử dụng nhiều kỹ thuật khác nhau nằm ngoài phạm vi của cuốn sách này. Sử dụng bộ giải QP là một cách để huấn luyện SVM. Một cách khác là sử dụng giảm độ dốc để tối thiểu hóa hàm mất mát bản lề (hinge loss) hoặc hàm mất mát bản lề bình phương (squared hinge loss) (xem Hình 5-13). Với một trường hợp x của lớp dương (tức là với $t = 1$), hàm mất mát là 0 nếu đầu ra s của hàm quyết định ($s = w^T x + b$) lớn hơn hoặc bằng 1. Điều này xảy ra khi trường hợp nằm ngoài “con đường” và ở phía dương. Với một trường hợp của lớp âm (tức là với $t = -1$), hàm mất mát là 0 nếu $s \leq -1$. Điều này xảy ra khi trường hợp nằm ngoài “con đường” và ở phía âm. Trường hợp càng xa phía đúng của biên thì hàm mất mát càng cao: nó tăng tuyến tính đối với hàm mất mát bản lề, và bậc hai đối với hàm mất mát bản lề bình phương. Điều này làm cho hàm mất mát bản lề bình phương nhạy cảm hơn với các ngoại lệ. Tuy nhiên, nếu tập dữ liệu sạch, nó có xu hướng hội tụ nhanh hơn. Theo mặc định, LinearSVC sử dụng hàm mất mát bản lề bình phương, trong khi SGDClassifier sử dụng hàm mất mát bản lề. Cả hai lớp đều cho phép bạn chọn hàm mất mát bằng cách đặt siêu tham số loss thành “hinge” hoặc “squared_hinge”. Thuật toán tối ưu hóa của lớp SVC tìm kiếm một giải pháp tương tự như việc tối thiểu hóa hàm mất mát bản lề.



Hình 5-13. Hàm mất mát bản lề (trái) và hàm mất mát bản lề bình phương (phải)

Tiếp theo, chúng ta sẽ xem xét một cách khác để huấn luyện bộ phân loại SVM tuyến tính: giải quyết bài toán đối ngẫu.

Bài toán đối ngẫu

Cho một bài toán tối ưu hóa có ràng buộc, được gọi là bài toán gốc (primal problem), có thể biểu diễn một bài toán khác nhưng có liên quan chặt chẽ, được gọi là bài toán đối ngẫu của nó. Giải pháp cho bài toán đối ngẫu thường đưa ra một cận dưới cho giải pháp của bài toán gốc, nhưng trong một số điều kiện, nó có thể có cùng giải pháp với bài toán gốc. May mắn thay, bài toán SVM đáp ứng các điều kiện này, vì vậy bạn có thể chọn giải bài toán gốc hoặc bài toán đối ngẫu; cả hai sẽ có cùng giải pháp.

Phương trình 5-3 cho thấy dạng đối ngẫu của mục tiêu SVM tuyến tính. Nếu bạn muốn biết cách dẫn xuất bài toán đối ngẫu từ bài toán gốc, hãy xem phần tài liệu bổ sung trong sổ tay của chương này. Phương trình 5-3. Dạng đối ngẫu của mục tiêu SVM tuyến tính tối thiểu hóa α

$$\underset{\alpha}{\text{minimize}} \quad \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha^{(i)} \alpha^{(j)} t^{(i)} t^{(j)} \mathbf{x}^{(i)T} \mathbf{x}^{(j)} - \sum_{i=1}^m \alpha^{(i)} \text{subject to } \alpha^{(i)} \geq 0$$

cho tất cả $i = 1, \dots, m$

Một khi bạn tìm thấy vector $\hat{\alpha}$ tối thiểu hóa phương trình này (sử dụng bộ giải QP), hãy sử dụng Phương trình 5-4 để tính toán $\hat{w}$ và $\hat{b}$ tối thiểu hóa bài toán gốc. Trong phương trình này, n_s biểu thị số lượng véc-tơ hỗ trợ. Phương trình 5-4. Từ giải pháp đối ngẫu đến giải pháp gốc

$$\mathbf{w} = \sum_{i=1}^m \alpha^{(i)} t^{(i)} \mathbf{x}^{(i)}$$

$$\hat{b} = \frac{1}{n_s} \sum_{\alpha^{(i)} > 0}^m (t^{(i)} - \hat{w}^T x^{(i)})$$

Bài toán đối ngẫu nhanh hơn để giải hơn bài toán gốc khi số lượng trường hợp huấn luyện nhỏ hơn số lượng đặc trưng. Quan trọng hơn, bài toán đối ngẫu giúp kernel trick trở nên khả thi, trong khi bài toán gốc thì không. Vậy kernel trick là gì?

SVM sử dụng Kernel (Kernelized SVMs)

Giả sử bạn muốn áp dụng phép biến đổi đa thức bậc hai lên một tập huấn luyện hai chiều, sau đó huấn luyện một bộ phân loại SVM tuyến tính trên tập dữ liệu đã được biến đổi. **Công thức 5-5** trình bày hàm ánh xạ đa thức bậc hai φ mà bạn muốn áp dụng.

Công thức 5-5: Ánh xạ đa thức bậc hai

$$\phi(\mathbf{x}) = \phi\left(\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}\right) = \begin{pmatrix} x_1^2 \\ \sqrt{2} x_1 x_2 \\ x_2^2 \end{pmatrix}$$

Lưu ý rằng vector đã biến đổi có 3 chiều thay vì 2. Bây giờ, hãy xem điều gì sẽ xảy ra với một cặp vector 2D, a và b , nếu chúng ta áp dụng ánh xạ đa thức bậc hai này rồi tính tích vô hướng của các vector đã biến đổi.

Công thức 5-6: Kỹ thuật Kernel cho ánh xạ đa thức bậc hai

$$\begin{aligned} \phi(\mathbf{a})^T \phi(\mathbf{b}) &= \begin{pmatrix} a_1^2 \\ \sqrt{2} a_1 a_2 \\ a_2^2 \end{pmatrix}^T \begin{pmatrix} b_1^2 \\ \sqrt{2} b_1 b_2 \\ b_2^2 \end{pmatrix} = a_1^2 b_1^2 + 2a_1 b_1 a_2 b_2 + a_2^2 b_2^2 \\ &= (a_1 b_1 + a_2 b_2)^2 = \left(\begin{pmatrix} a_1 \\ a_2 \end{pmatrix}^T \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} \right)^2 = (\mathbf{a}^T \mathbf{b})^2 \end{aligned}$$

Tích vô hướng của các vector đã biến đổi bằng với bình phương của tích vô hướng của các vector ban đầu. Đây là mấu chốt quan trọng: nếu bạn áp dụng phép biến đổi φ cho tất cả các trường hợp huấn luyện, thì bài toán đối ngẫu sẽ chứa tích vô hướng $\varphi(x^{(i)})^T \varphi(x^{(j)})$. Nhưng nếu φ là phép biến đổi đa thức bậc hai được định nghĩa trong Công thức 5-5, thì bạn có thể thay thế tích vô hướng của các vector đã biến đổi bằng $(x^{(i)T} x^{(j)})^2$. Vì vậy, bạn không cần phải biến đổi tập huấn luyện; chỉ cần thay thế tích vô hướng bằng bình phương của nó. Kết quả sẽ giống hệt như khi bạn đã thực hiện toàn bộ quá trình biến đổi tập dữ liệu và sau đó huấn luyện thuật toán SVM tuyến tính, nhưng cách này hiệu quả hơn về mặt tính toán.

Trong học máy, một hàm có khả năng tính toán tích vô hướng $\varphi(a)^T \varphi(b)$, chỉ dựa trên các vector gốc a và b mà không cần phải tính toán phép biến đổi φ , được gọi là **kernel**. **Công thức 5-7** liệt kê một số kernel thường được sử dụng nhất.

Công thức 5-7: Các Kernel thông dụng

- **Tuyến tính:** $K(a, b) = a^T b$
- **Đa thức:** $K(a, b) = (\gamma a^T b + r)^d$
- **RBF Gauss:** $K(a, b) = \exp(-\gamma \|a - b\|^2)$
- **Sigmoid:** $K(a, b) = \tanh(\gamma a^T b + r)$

Vẫn còn một vấn đề cần giải quyết. **Công thức 5-4** cho thấy cách chuyển từ nghiệm đối ngẫu sang nghiệm sơ cấp trong trường hợp bộ phân loại SVM tuyến tính. Nhưng nếu bạn áp dụng kỹ thuật kernel, bạn sẽ có các phương trình bao gồm $\varphi(x^{(i)})$. Trên thực tế, φ phải có cùng số chiều với $\varphi(x^{(i)})$, có thể rất lớn hoặc thậm chí vô hạn, nên bạn không thể tính toán nó. Nhưng làm thế nào bạn có thể đưa ra dự đoán mà không biết φ ? Tin tốt là bạn có thể thay công thức của φ từ Công thức 5-4 vào hàm quyết định cho một trường hợp mới $x^{(n)}$, và bạn sẽ có một phương trình chỉ chứa các tích vô hướng giữa các vector đầu vào. Điều này giúp bạn có thể sử dụng kỹ thuật kernel.

Công thức 5-8: Đưa ra dự đoán bằng Kernelized SVM

$$\begin{aligned} h_{\hat{\mathbf{w}}, \hat{b}}(\phi(\mathbf{x}^{(n)})) &= \hat{\mathbf{w}}^T \phi(\mathbf{x}^{(n)}) + \hat{b} = \left(\sum_{i=1}^m \hat{\alpha}^{(i)} t^{(i)} \phi(\mathbf{x}^{(i)}) \right)^T \phi(\mathbf{x}^{(n)}) + \hat{b} \\ &= \sum_{i=1}^m \hat{\alpha}^{(i)} t^{(i)} \left(\phi(\mathbf{x}^{(i)})^T \phi(\mathbf{x}^{(n)}) \right) + \hat{b} \\ &= \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \hat{\alpha}^{(i)} t^{(i)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(n)}) + \hat{b} \end{aligned}$$

Lưu ý rằng vì $\alpha^{(i)} \neq 0$ chỉ đối với các vector hỗ trợ, việc đưa ra dự đoán chỉ liên quan đến việc tính tích vô hướng của vector đầu vào mới $x^{(n)}$ với các vector hỗ trợ, chứ không phải với tất cả các trường hợp huấn luyện. Tất nhiên, bạn cũng phải sử dụng cùng một kỹ thuật để tính số hạng độ lệch b .

Công thức 5-9: Sử dụng kỹ thuật Kernel để tính số hạng độ lệch

$$\begin{aligned}\hat{\mathbf{b}} &= \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left(t^{(i)} - \hat{\mathbf{w}}^\top \phi(\mathbf{x}^{(i)}) \right) = \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left(t^{(i)} - \left(\sum_{j=1}^m \hat{\alpha}^{(j)} t^{(j)} \phi(\mathbf{x}^{(j)}) \right)^\top \phi(\mathbf{x}^{(i)}) \right) \\ &= \frac{1}{n_s} \sum_{\substack{i=1 \\ \hat{\alpha}^{(i)} > 0}}^m \left(t^{(i)} - \sum_{\substack{j=1 \\ \hat{\alpha}^{(j)} > 0}}^m \hat{\alpha}^{(j)} t^{(j)} K(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \right)\end{aligned}$$

Nếu bạn cảm thấy đau đầu, điều đó hoàn toàn bình thường: đây là một tác dụng phụ không may của kỹ thuật kernel.

Bài tập

1. Ý tưởng cơ bản đằng sau máy học véc-tơ hỗ trợ là gì?
2. Véc-tơ hỗ trợ là gì?
3. Tại sao việc chuẩn hóa đầu vào lại quan trọng khi sử dụng SVM?
4. Một bộ phân loại SVM có thể đưa ra điểm tin cậy khi phân loại một trường hợp không? Còn xác suất thì sao?
5. Làm thế nào để bạn chọn giữa LinearSVC, SVC và SGDClassifier?
6. Giả sử bạn đã huấn luyện một bộ phân loại SVM với kernel RBF, nhưng nó dường như bị dưới khớp trên tập huấn luyện. Bạn nên tăng hay giảm γ (gamma)? Còn C thì sao?
7. Một mô hình được gọi là không nhạy cảm với ϵ có nghĩa là gì?
8. Mục đích của việc sử dụng kernel trick là gì?
9. Huấn luyện một LinearSVC trên một tập dữ liệu có thể phân tách tuyến tính. Sau đó huấn luyện một SVC và một SGDClassifier trên cùng tập dữ liệu đó. Xem liệu bạn có thể làm cho chúng tạo ra cùng một mô hình hay không.
10. Huấn luyện một bộ phân loại SVM trên tập dữ liệu wine, bạn có thể tải bằng cách sử dụng `sklearn.datasets.load_wine()`. Tập dữ liệu này chứa các phân tích hóa học của 178 mẫu rượu được sản xuất bởi 3 nhà sản xuất khác nhau: mục tiêu là huấn luyện một mô hình phân loại có khả năng dự đoán nhà sản xuất dựa trên phân tích hóa học của rượu. Vì bộ phân loại SVM là bộ phân loại nhị phân, bạn sẽ cần sử dụng chiến lược “một đối tất cả” (one-versus-all) để phân loại cả ba lớp. Bạn có thể đạt được độ chính xác bao nhiêu?
11. Huấn luyện và tinh chỉnh một hồi quy SVM trên tập dữ liệu California housing. Bạn có thể sử dụng tập dữ liệu gốc thay vì phiên bản đã được điều chỉnh mà chúng ta đã sử dụng trong Chương 2, bạn có thể tải bằng cách sử dụng `sklearn.datasets.fetch_california_housing()`. Các mục tiêu đại diện cho hàng trăm nghìn đô la. Vì có hơn 20.000 trường hợp, SVM có thể chậm, vì vậy để tinh chỉnh siêu tham số, bạn nên sử dụng ít trường hợp hơn (ví dụ: 2.000) để kiểm tra nhiều kết hợp siêu tham số hơn. RMSE của mô hình tốt nhất của bạn là bao nhiêu? Các giải pháp

cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.